

Objectives

- To implement the Apriori algorithm for extracting association rules from the engineered Co-Occurrence and Price Spike transaction baskets.
- To diagnose and resolve the severe memory consumption issues inherent to candidate generation on large-scale agricultural datasets.
- To establish a mathematical baseline of association rules to benchmark against the more advanced FP-Growth algorithm.
- To identify initial supply chain bundling and localized inflation clusters under constrained parameters.

Tasks Completed Algorithm Optimization Engineer (Carelyn Macabuhay)

- **Tasks:** Apriori implementation, memory management, and parameter tuning.
- **Description:**
 - Deployed the `apriori` function from `mlxtend.frequent_patterns` to process the boolean-encoded transaction datasets.
 - Diagnosed a critical algorithmic bottleneck: the system threw a `MemoryError` because NumPy attempted to allocate an unmanageable ~105 GiB boolean tensor during candidate generation.
 - Engineered a "memory-safe" sandboxed execution environment by strictly downsampling the data: capped transactions at 15,000 rows, limited item columns to the top 80, set `max_len=3`, and utilized the `low_memory=True` flag to prevent matrix explosion.
 - Successfully generated a baseline of 124,609 co-occurrence rules (support ≥ 0.15) and 36 price spike rules (support ≥ 0.20) under these strict boundaries.
- **Screenshot/Code Snippet:**

Python

```
# == Apriori-specific tighter parameters to prevent MemoryError ==  
  
MIN_SUP_CO_AP = 0.15 # tighter support for Apriori co-occ  
  
TOP_ITEMS_AP = 80 # hard column cap (peak mem stays under 2 GB)  
  
MAX_ROWS_AP = 15_000 # row cap for Apriori dense boolean matrix  
  
  
# Running the constrained algorithm  
  
co_freq_ap = apriori(co_samp,  
                     min_support=MIN_SUP_CO_AP,  
                     use_colnames=True,  
                     max_len=3,  
                     low_memory=True)
```

Tasks to be Completed

- Filter the extracted Apriori baseline rules to see if localized, region-specific datasets (e.g., isolating only ARMM data) allow the algorithm to run without aggressive row downsampling.
- Draft the algorithmic methodology section for the final manuscript, detailing the mathematical reasons behind Apriori's memory failure and justifying the team's pivot to FP-Growth.

Results and Analysis

1. **Algorithmic Limits & Performance:** My phase proved that while Apriori recorded a technically "fast" runtime (1.31s for Co-Occurrence, 0.02s for Price Spikes), this speed was entirely artificial. It only succeeded because I handicapped the algorithm to a fraction of the actual data. This established that Apriori is practically unusable for the full 2.5 million-row dataset without sacrificing deep, low-frequency patterns.
2. **Baseline Supply Chain Findings:** Despite the strict constraints, the Apriori model successfully identified highly confident premium physical supply chain bundles. For example, the presence of "Crab" and "Rice (premium)" in a market highly predicts the presence of "Shrimp (endeavor)" with a Lift of 2.88, confirming consistent retail behaviors for higher-end commodities.
3. **Vegetable Inflation Clusters:** The constrained Apriori model successfully mapped localized price spike behaviors, specifically within vegetables. I found that when "Carrots" and "Tomatoes" experience a price spike (hitting their 75th percentile), there is an 88.6% confidence that "Cabbage" is also spiking simultaneously (Lift: 2.29).

Challenges and Solutions

- **Challenge:** The primary challenge was the catastrophic memory failure. The Apriori algorithm's level-by-level candidate generation creates an exponentially massive combination of itemsets, causing the Python environment to completely crash when attempting to process the 46,591 transactions built by the Data Architect.
- **Solution:** I implemented a custom `encode_apriori` pipeline. By enforcing a hard column cap (`top_n=80`) and a row sample limit (`max_rows=15,000`), I reduced the required memory overhead to a flat ~1 MB. This "sandbox" approach safely extracted the baseline rules without crashing the team's workflow.

Conclusion My implementation successfully demonstrated the theoretical utility, but practical limitations, of the Apriori algorithm for big agricultural data. While the algorithm generated highly interpretable and logical rules regarding food bundling and vegetable inflation clusters, its memory-intensive nature makes it incapable of scaling. By securely diagnosing this bottleneck and establishing a working baseline, I provided the necessary technical justification for the ML Engineer to deploy the tree-based FP-Growth algorithm for the project's final comprehensive analysis.